

CS T34-DIGITAL SYSEM DESIGN

UNIT – II (11MARKS)

Y2/S3

UNIT II

Combinational Logic: Half & full adders/subtractors – Parallel Adders – Look-ahead carry adders - BCD adders/subtractors – Binary Multiplier – Code convertors – Decoders – Encoders – Parity encoders – Multiplexers – Implementation of combinational logic using Multiplexers - Demultiplexers – Magnitude comparators – Parity generator/checker.

1. Explain the logic design of a Full adder. (5) (NOV'11)
Discuss about Half adder and Full adder with diagram. (11) (APR'13)
Explain the full adder in detail. (11) (APR'13 - IT)
Explain in detail half adder and full subtractor. (7) (APR'10)

(a) HALF ADDERS

- A combinational circuit that performs the addition of two bits is called a *half adder*.
- The variables x and y to the two inputs and S (for sum) and C (for carry) to the outputs.
- The truth table for the half adder is listed in Table.
- The C output is 1 only when both inputs are 1.
- The S output represents the least significant bit of the sum.

Truth Table For Half Adder

X	Y	S(Sum)	C(Carry)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

- The simplified Boolean functions for the two outputs can be obtained directly from the truth table.
- The simplified sum-of-products expressions are

$$S = x'y + xy'$$
$$C = xy$$

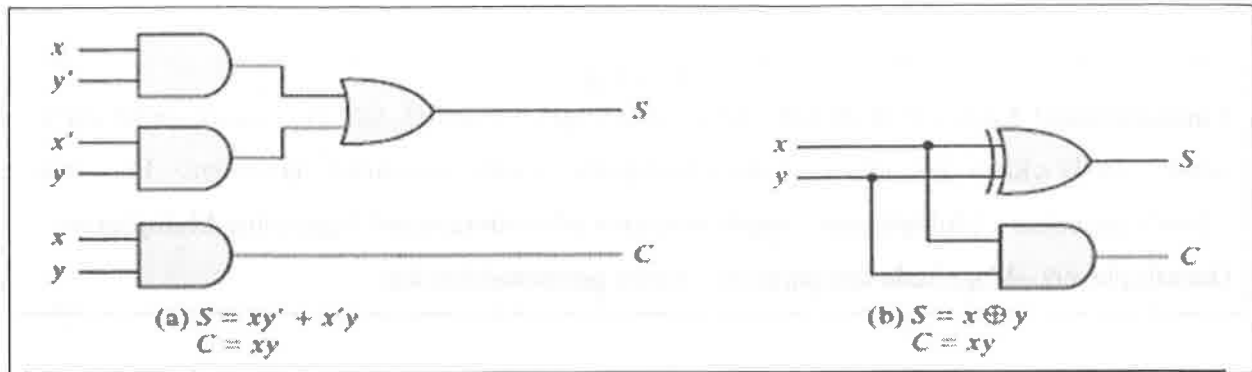


Figure : Implementation Of Half Adder

(b) FULL ADDERS

- *A full adder is a combinational circuit that forms the arithmetic sum of three bits.*
- It consists of three inputs and two outputs. Two of the input variables, denoted by x and y , represent the two significant bits to be added.
- The third input, z , represents the carry from the previous lower significant position.
- Two outputs are necessary because the arithmetic sum of three binary digits ranges in value from 0 to 3, and binary representation of 2 or 3 needs two bits. The two outputs are designated by the symbols S for sum and C for carry.
- The truth table of the full adder is listed in Table.

Truth Table For Full Adder

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

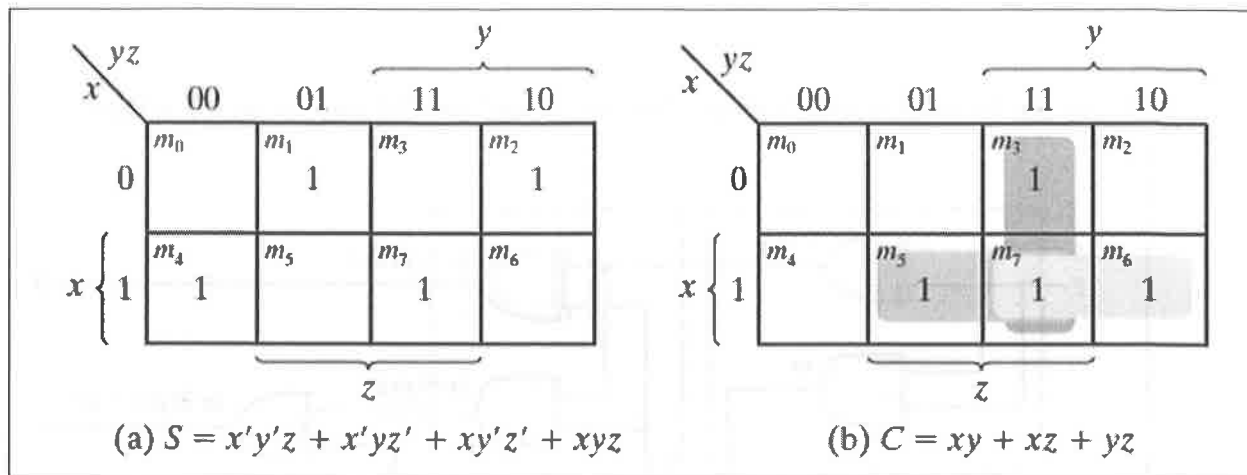


Figure 1 : K-Maps for full adder

- The maps for the outputs of the full adder are shown in Fig.1. The simplified expressions are

$$S = x'y'z + x'yz' + xy'z' + xyz$$

$$C = xy + xz + yz$$

- The logic diagram for the full adder implemented in sum-of-products form is given below

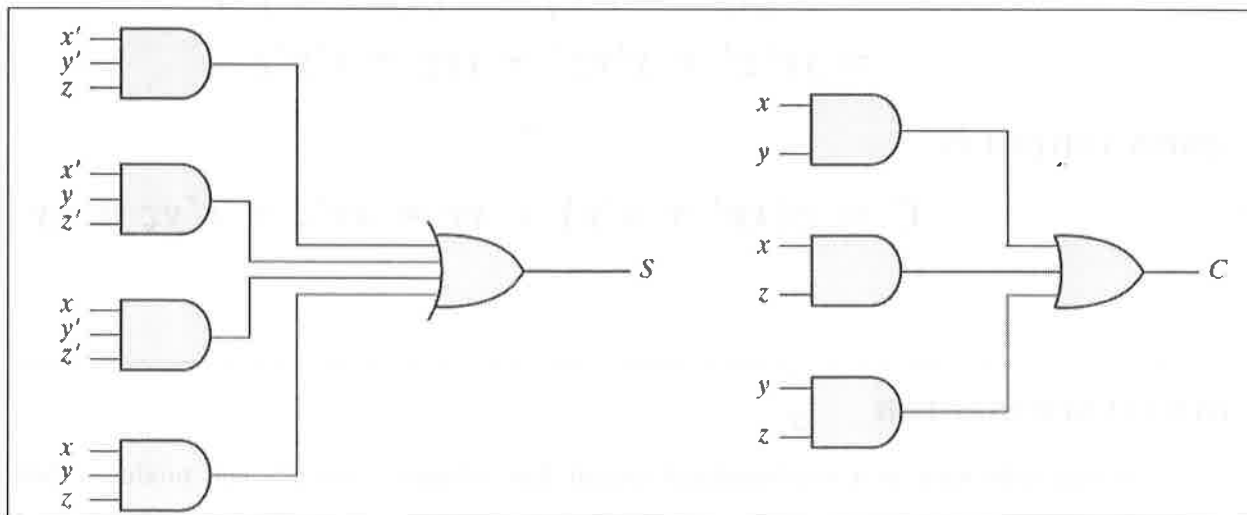


Figure 2: Implementation of full adder in sum-of-products form

- It can also be implemented with two half adders and one OR gate, as shown in figure 3.

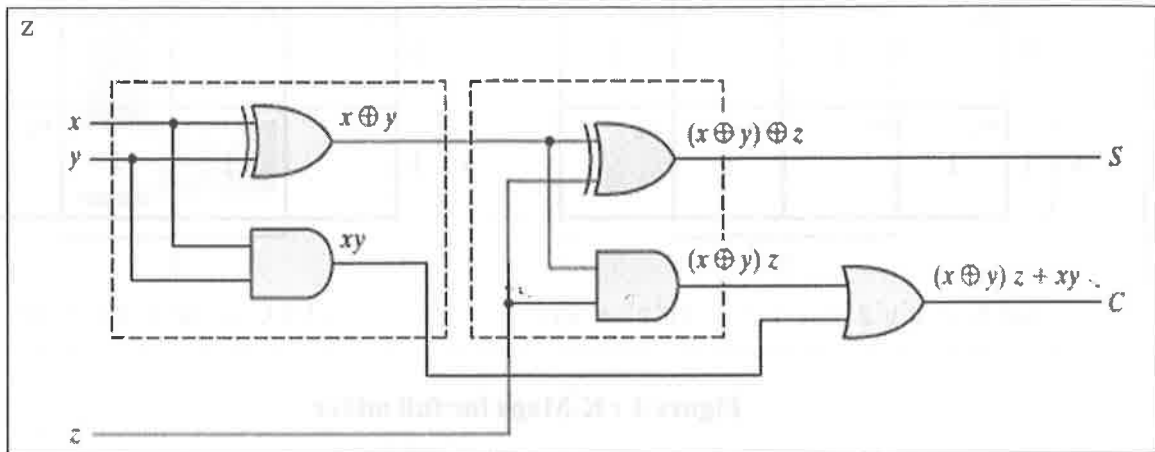


Figure 3: Implementation of full adder with two half adders and an OR gate

- The S output from the second half adder is the exclusive-OR of z and the output of the first half adder, giving

$$\begin{aligned}
 S &= z \oplus (x \oplus y) \\
 &= z'(xy' + x'y) + z(xy' + x'y)' \\
 &= z'(xy' + x'y) + z(xy + x'y') \\
 &= xy'z' + x'yz' + xyz + x'y'z
 \end{aligned}$$

carry output is

$$C = z(xy' + x'y) + xy = xy'z + x'yz + xy$$

(c) HALF SUBTRACTOR

- A half-subtractor is a combinational circuit that subtracts two bits and produces their difference.
- It also has an output to specify if a 1 has been borrowed.
- Designate the minuend bit by X and the subtrahend bit by y.
- To perform $x - y$, we have to check the relative magnitudes of x and y.
- If $x < y$, we have three possibilities: $0 - 0 = 0$, $1 - 0 = 1$, and $1 - 1 = 0$.

- The result is called the *difference bit*. One output generates the difference and will be designated by the symbol D .
- The second output, designated B for borrow, generates the binary signal that informs the next stage that a 1 has been borrowed.
- The truth table for the input-output relationships of a half-subtractor can now be derived as follows:

Truth Table For Half Subtractor

X	Y	D(Difference)	B(Borrow)
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

- The output borrow B is a 0 as long as $x \geq y$. It is a 1 for $x = 0$ and $y = 1$.
- The D output is the result of the arithmetic operation $2B + x - y$.
- The Boolean functions for the two outputs of the half-subtractor are derived directly from the truth table:

$$D = X'Y + XY'$$
$$B = X'Y$$

- It is interesting to note that the logic for D is exactly the same as the logic for output S in the half-adder.

(d) FULL SUBTRACTOR

- A full-subtractor is a combinational circuit that performs a subtraction between two bits, taking into account that a 1 may have been borrowed by a lower significant stage.
- This circuit has three inputs and two outputs. The three inputs, x , y , and z , denote the minuend, subtrahend, and previous borrow, respectively.
- The two outputs, D and B , represent the difference and output borrow, respectively.
- The truth table for the circuit is

CS T34-DIGITAL SYSEM DESIGN

UNIT – II (11MARKS)

Y2/S3

Truth Table For Full Subtractor

INPUT			OUTPUT	
X	Y	Z	D	B
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

- The simplified Boolean functions for the two outputs of the full-subtractor are derived in the maps of Fig. 1.
- The simplified sum of products output functions are

$$D = x'y'z + x'yz' + xy'z' + xyz$$

$$B = x'y + x'z + yz$$

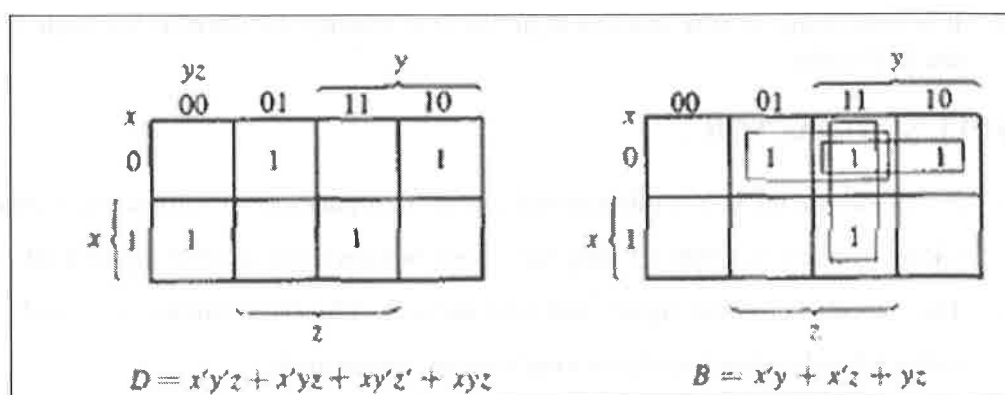


Figure 1: maps for full subtractor

2. Explain in detail the 4 bit binary parallel adder. (NOV'12)

- A binary parallel adder is a digital circuit that produces the arithmetic sum of two binary numbers in parallel.
- It consists of full-adders connected in a chain, with the output carry from each full-adder connected to the input carry of the next full-adder in the chain.
- When the 4-bit full-adder circuit is enclosed within an IC package, it has four terminals for the augend bits, four terminals for the addend bits, four terminals for the sum bits, and two terminals for the input and output carries.
- Addition of n -bit numbers requires a chain of n full adders or a chain of one-half adder and $n - 1$ full adders.
- Figure shows the interconnection of four full-adder (FA) circuits to provide a four-bit binary ripple carry adder.

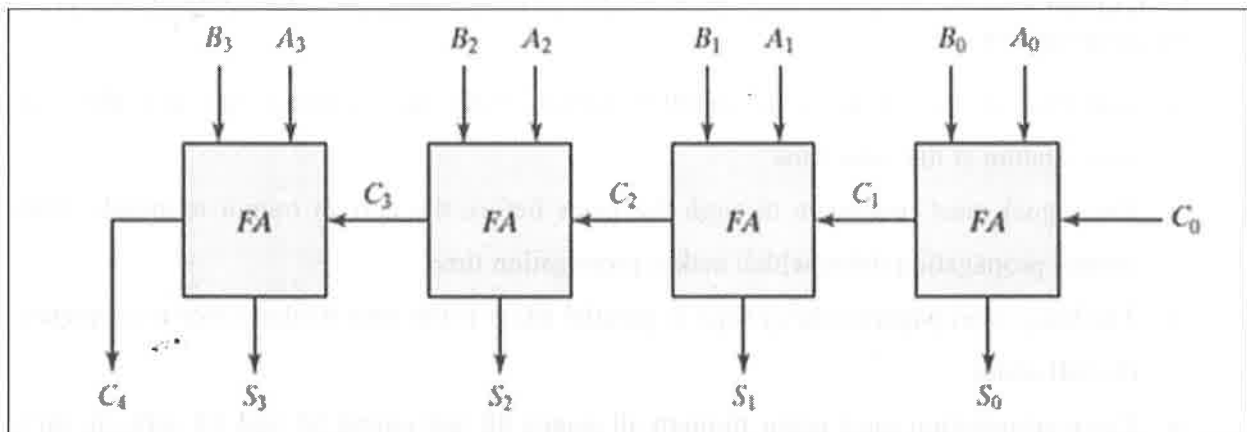


Figure: 4-Bit Binary Parallel Adder

- The augend bits of A and the addend bits of B are designated by subscript numbers from right to left, with subscript 0 denoting the least significant bit.
- The carries are connected in a chain through the full adders. The input carry to the adder is C_0 , and it ripples through the full adders to the output carry C_4 .
- The S outputs generate the required sum bits.
- An n -bit adder requires n full adders, with each output carry connected to the input carry of the next higher order full adder.
- To demonstrate with a specific example, consider the two binary numbers $A = 1011$ and $B = 0011$. Their sum $S = 1110$ is formed with the four-bit adder as follows:

Subscript i :	3	2	1	0	
Input carry	0	1	1	0	C_i
Augend	1	0	1	1	A_i
Addend	0	0	1	1	B_i
Sum	1	1	1	0	S_i
Output carry	0	0	1	1	C_{i+1}

- Observe that the design of this circuit by the classical method would require a truth table with $2^9 = 512$ entries, since there are nine inputs to the circuit.
- By using an iterative method of cascading an already known function, we were able to obtain a simple and well-organized implementation.

3. Explain with the help of a logic diagram the principle of a carry look ahead adder.

Carry propagation

- Addition of two number in parallel implies both the numbers are available for computation at the same time.
- The signal must propagate through the gates before the correct output terminals. This creates propagation delay which makes propagation time.
- The longest propagation delay time in parallel adder is the time it takes carry to propagate the full adder.
- Carry propagation must occur through all stages till last output S_3 and C_4 settle to their “final steady state value”.

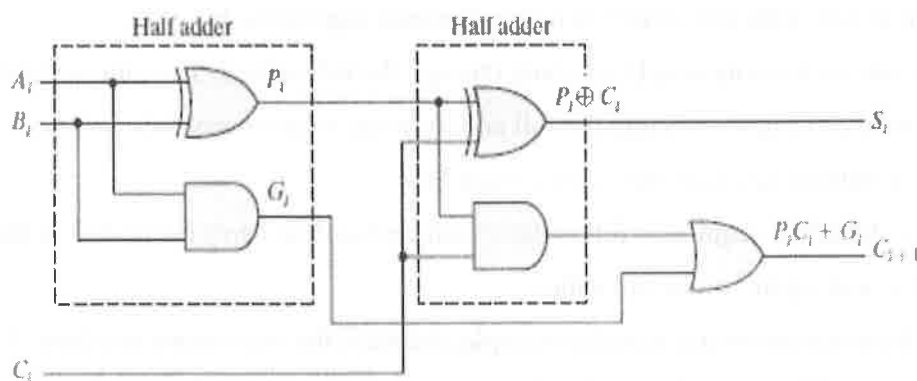


Figure1: Full adder with P and G

- If we define two new binary variables

$$P_i = A_i \oplus B_i$$

$$G_i = A_i B_i$$

- the output sum and carry can respectively be expressed as

$$S_i = P_i \oplus C_i$$

$$C_{i+1} = G_i + P_i C_i$$

- G_i is called a *carry generate*, and it produces a carry of 1 when both A_i and B_i are 1, regardless of the input carry C_i . P_i is called a *carry propagate*, because it determines whether a carry into stage i will propagate into stage $i + 1$.

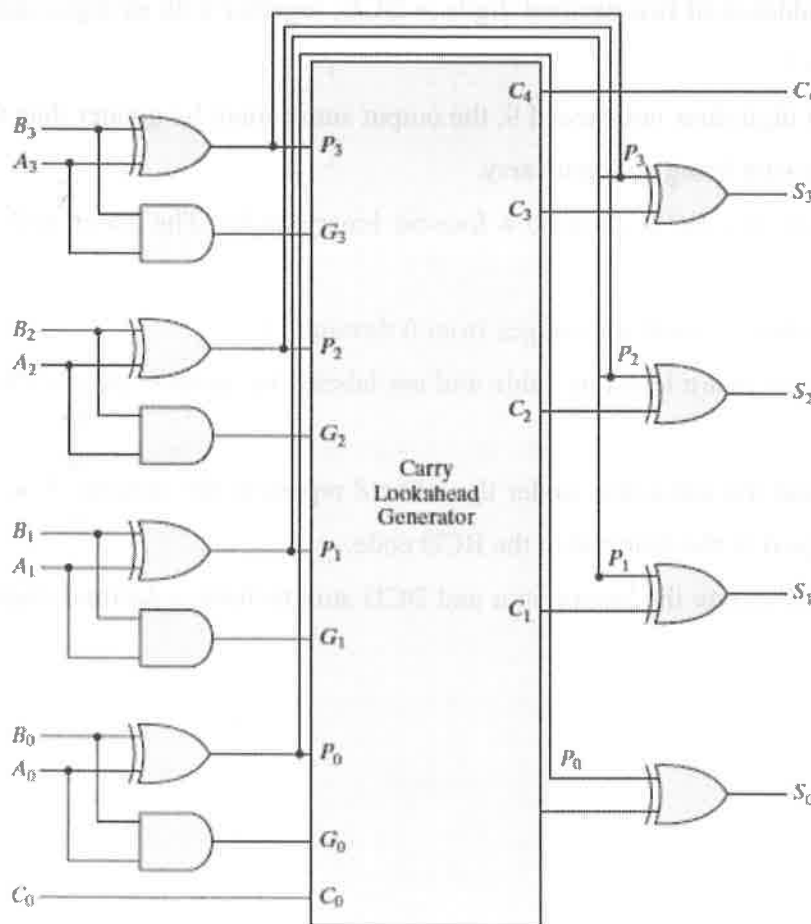


Figure2: Four-Bit Adder with a Carry Lookahead

- The construction of a four-bit adder with a carry lookahead scheme is shown in Fig.2
- Each sum output requires two exclusive-OR gates.
- The output of the first exclusive-OR gate generates the P_i variable, and the AND gate generates the G_i variable.

- The carries are propagated through the carry lookahead generator (similar to that in Fig.1) and applied as inputs to the second exclusive-OR gate.
- All output carries are generated after a delay through two levels of gates.
- Thus, outputs $S1$ through $S3$ have equal propagation delay times.
- The two-level circuit for the output carry $C4$ is not shown.
- This circuit can easily be derived by the equation-substitution method.

4. With a neat diagram explain BCD adder.	(11 mark)(NOV'11)
--	--------------------------

- The arithmetic addition of two decimal digits in BCD, together with an input carries from a previous stage.
- Since each input digit does not exceed 9, the output sum cannot be greater than $9 + 9 + 1 = 19$, the 1 in the sum being an input carry.
- Suppose we apply two BCD digits to a four-bit binary adder. The adder will form the sum in *binary* and produce a result that ranges from 0 through 19.
- These binary numbers are listed in Table and are labeled by symbols K , $Z8$, $Z4$, $Z2$, and $Z1$.
- K is the carry, and the subscripts under the letter Z represent the weights 8, 4, 2, and 1 that can be assigned to the four bits in the BCD code.
- Tabular column illustrate the binary sum and BCD sum to form a decimal digits of 0 to 19.

CS T34-DIGITAL SYSEM DESIGN

UNIT - II (11MARKS)

Y2/S3

Derivation of BCD Adder

K	Binary Sum				BCD Sum					Decimal
	Z ₈	Z ₄	Z ₂	Z ₁	C	S ₈	S ₄	S ₂	S ₁	
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	2
0	0	0	1	1	0	0	0	1	1	3
0	0	1	0	0	0	0	1	0	0	4
0	0	1	0	1	0	0	1	0	1	5
0	0	1	1	0	0	0	1	1	0	6
0	0	1	1	1	0	0	1	1	1	7
0	1	0	0	0	0	1	0	0	0	8
0	1	0	0	1	0	1	0	0	1	9
0	1	0	1	0	1	0	0	0	0	10
0	1	0	1	1	1	0	0	0	1	11
0	1	1	0	0	1	0	0	1	0	12
0	1	1	0	1	1	0	0	1	1	13
0	1	1	1	0	1	0	1	0	0	14
0	1	1	1	1	1	0	1	0	1	15
1	0	0	0	0	1	0	1	1	0	16
1	0	0	0	1	1	0	1	1	1	17
1	0	0	1	0	1	1	0	0	0	18
1	0	0	1	1	1	1	0	0	1	19

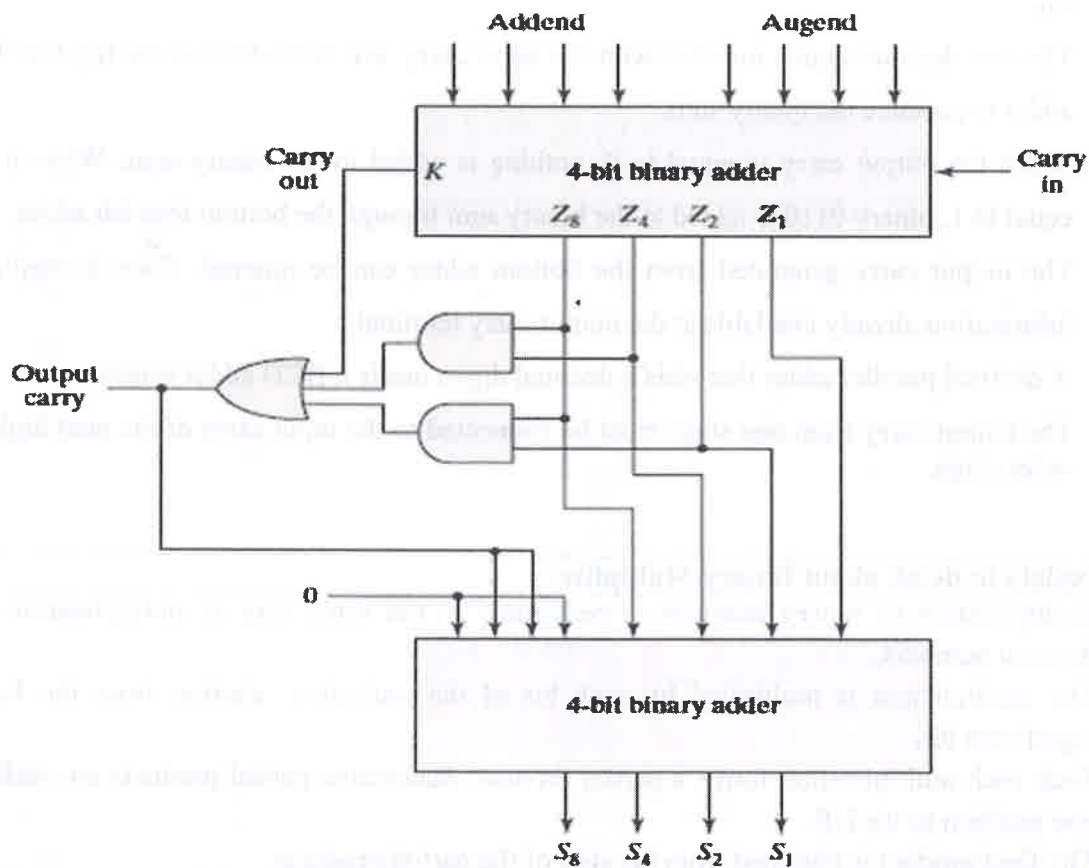


Figure: BCD ADDER

- The logic circuit that detects the necessary correction can be derived from the entries in the table.
- It is obvious that a correction is needed when the binary sum has an output carry $K = 1$.
- The other six combinations from 1010 through 1111 that need a correction have a 1 in position Z8.
- To distinguish them from binary 1000 and 1001, which also have a 1 in position Z8, we specify further that either Z4 or Z2 must have a 1.
- The condition for a correction and an output carry can be expressed by the Boolean function

$$C = K + Z8Z4 + Z8Z2$$

- When $C = 1$, it is necessary to add 0110 to the binary sum and provide an output carry for the next stage.
- A BCD adder that adds two BCD digits and produces a sum digit in BCD is shown in Fig..
- The two decimal digits, together with the input carry, are first added in the top four-bit adder to produce the binary sum.
- When the output carry is equal to 0, nothing is added to the binary sum. When it is equal to 1, binary 0110 is added to the binary sum through the bottom four-bit adder.
- The output carry generated from the bottom adder can be ignored, since it supplies information already available at the output carry terminal.
- A decimal parallel adder that adds n decimal digits needs n BCD adder stages.
- The output carry from one stage must be connected to the input carry of the next higher order stage.

5. Explain in detail about Binary Multiplier.

- Multiplication of binary numbers is performed in the same way as multiplication of decimal numbers.
- The multiplicand is multiplied by each bit of the multiplier, starting from the least significant bit.
- Each such multiplication forms a partial product. Successive partial products are shifted one position to the left.
- The final product is obtained from the sum of the partial products.
- A binary multiplier can be implemented with a combinational circuit, consider the multiplication of two 2-bit numbers as shown in Fig.

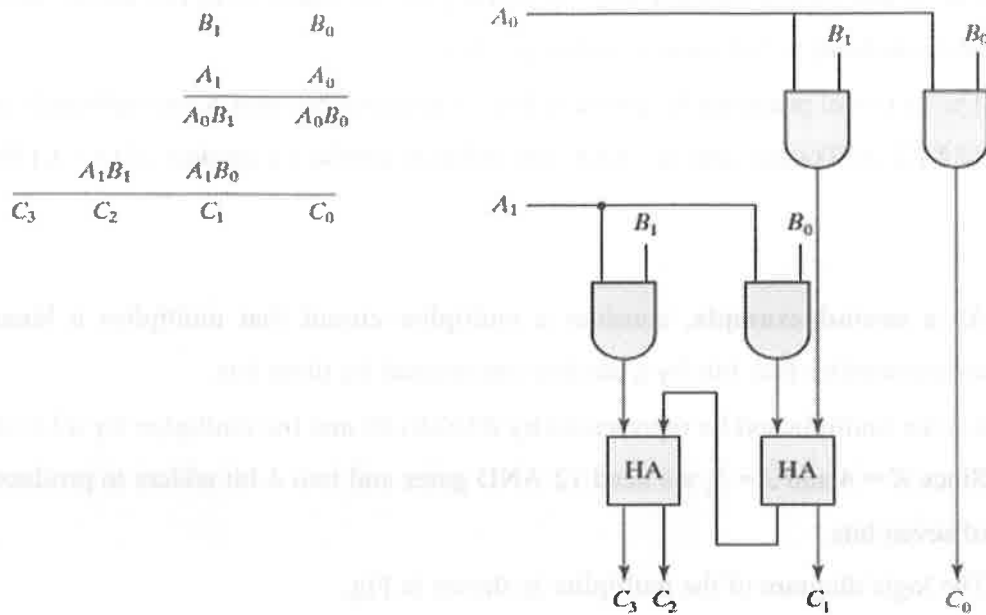


Figure: Two-bit by two-bit binary multiplier

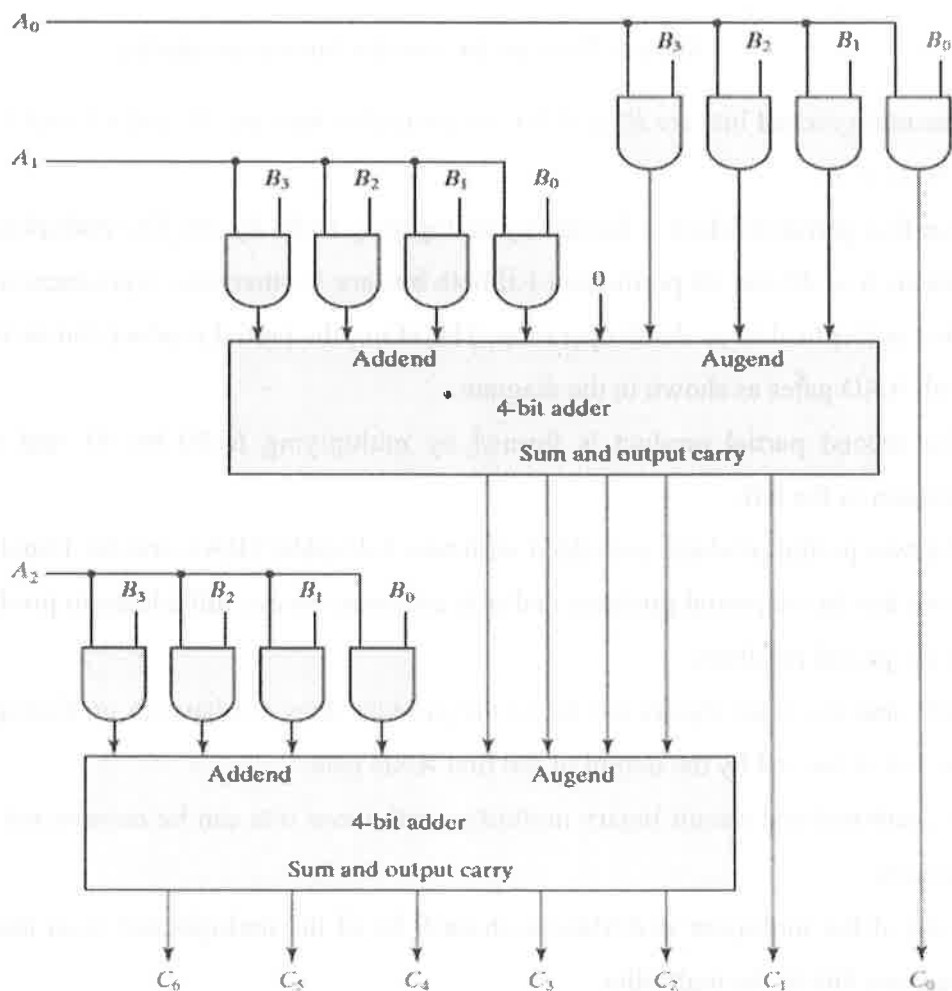
- The multiplicand bits are B_1 and B_0 , the multiplier bits are A_1 and A_0 , and the product is $C_3C_2C_1C_0$.
- The first partial product is formed by multiplying B_1B_0 by A_0 . The multiplication of two bits such as A_0 and B_0 produces a 1 if both bits are 1; otherwise, it produces a 0.
- This is identical to an AND operation. Therefore, the partial product can be implemented with AND gates as shown in the diagram.
- The second partial product is formed by multiplying B_1B_0 by A_1 and shifting one position to the left.
- The two partial products are added with two half-adder (HA) circuits. Usually, there are more bits in the partial products and it is necessary to use full adders to produce the sum of the partial products.
- Note that the least significant bit of the product does not have to go through an adder, since it is formed by the output of the first AND gate.
- A combinational circuit binary multiplier with more bits can be constructed in a similar fashion.
- A bit of the multiplier is ANDed with each bit of the multiplicand in as many levels as there are bits in the multiplier.

CS T34-DIGITAL SYSEM DESIGN

UNIT – II (11MARKS)

Y2/S3

- The binary output in each level of AND gates is added with the partial product of the previous level to form a new partial product.
- The last level produces the product. For J multiplier bits and K multiplicand bits, we need $1J * K2$ AND gates and $1J - 12 K$ -bit adders to produce a product of $(J + K)$ bits.
- As a **second example**, consider a multiplier circuit that multiplies a binary number represented by four bits by a number represented by three bits.
- Let the multiplicand be represented by $B_3B_2B_1B_0$ and the multiplier by $A_2A_1A_0$.
- Since $K = 4$ and $J = 3$, we need 12 AND gates and two 4-bit adders to produce a product of seven bits.
- The logic diagram of the multiplier is shown in Fig.



6. Explain in detail about Decoders and Encoders.

Explain the decoders in detail. (11) (APR'13)

Write in detail about Decoders and Encoders with circuit diagram. (11)(APR/MAY'12)

- A *decoder* is a combinational circuit that converts binary information from n input lines to a maximum of 2^n unique output lines.
- If the n -bit coded information has unused combinations, the decoder may have fewer than 2^n outputs.
- The decoder's presented here are called n -to- m -line decoders, where $m \leq 2^n$.
- Their purpose is to generate the 2^n (or fewer) minterms of n input variables.
- Each combination of inputs will assert a unique output.
- The name *decoder* is also used in conjunction with other code converters, such as a BCD-to-seven-segment decoder.
- As an example, consider the three-to-eight-line decoder circuit of Figure.

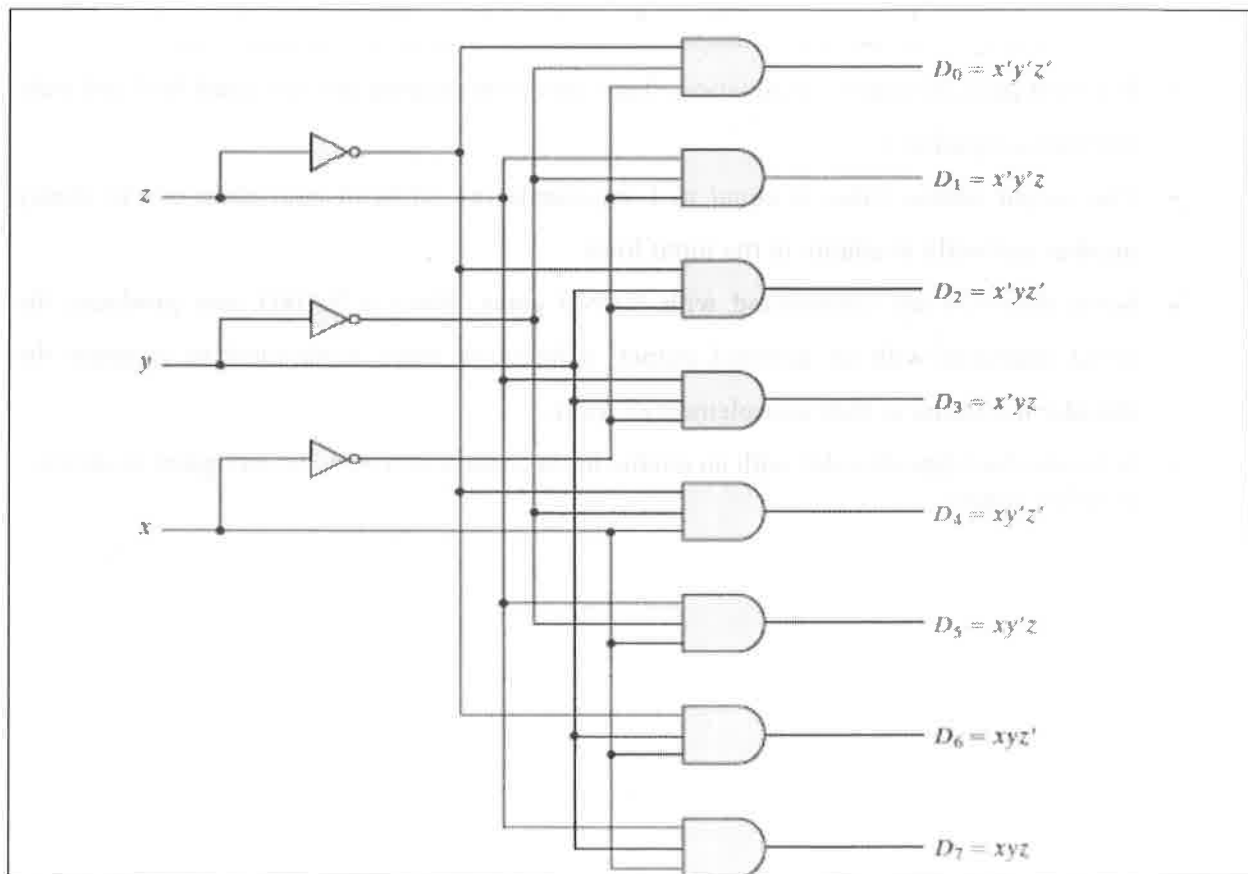


Figure: Three-to-eight-line decoder

CS T34-DIGITAL SYSEM DESIGN

UNIT – II (11MARKS)

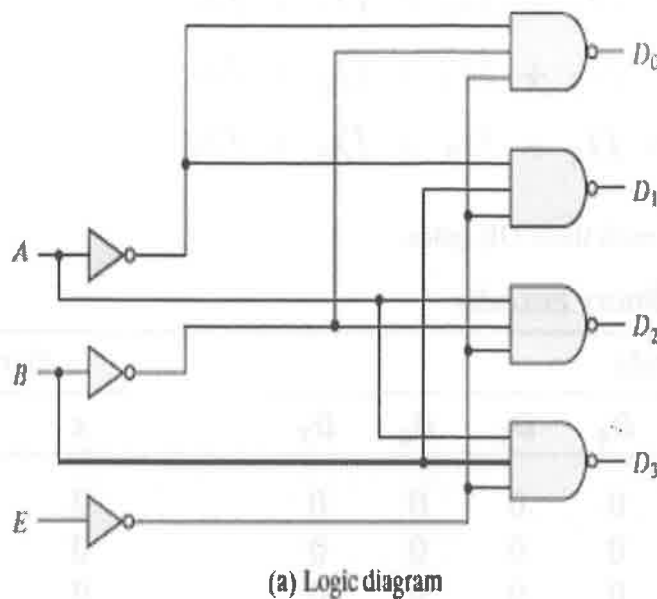
Y2/S3

- The three inputs are decoded into eight outputs, each representing one of the minterms of the three input variables.
- The three inverters provide the complement of the inputs, and each one of the eight AND gates generate one of the minterms.
- The operation of the decoder may be clarified by the truth table listed in Table.

Truth Table of a Three-to-Eight-Line Decoder

Inputs			Outputs							
x	y	z	D ₀	D ₁	D ₂	D ₃	D ₄	D ₅	D ₆	D ₇
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

- For each possible input combination, there are seven outputs that are equal to 0 and only one that is equal to 1.
- The output whose value is equal to 1 represents the minterm equivalent of the binary number currently available in the input lines.
- Some decoders are constructed with NAND gates. Since a NAND gate produces the AND operation with an inverted output, it becomes more economical to generate the decoder minterms in their complemented form.
- A two-to-four-line decoder with an enable input constructed with NAND gates is shown in below figure.



<i>E</i>	<i>A</i>	<i>B</i>	<i>D</i> ₀	<i>D</i> ₁	<i>D</i> ₂	<i>D</i> ₃
1	X	X	1	1	1	1
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	1

(b) Truth table

Figure: Two-to-four-line decoder with enable input

ENCODER

- An encoder is a digital circuit that performs the inverse operation of a decoder. An encoder has $2n$ (or fewer) input lines and n output lines.
- The output lines, as an aggregate, generate the binary code corresponding to the input value.
- An example of an encoder is the octal-to-binary encoder whose truth table is given in Table 4.7.
- It has eight inputs (one for each of the octal digits) and three outputs that generate the corresponding binary number. It is assumed that only one input has a value of 1 at any given time.
- The encoder can be implemented with OR gates whose inputs are determined directly from the truth table.
- Output z is equal to 1 when the input octal digit is 1, 3, 5, or 7. Output y is 1 for octal digits 2, 3, 6, or 7, and output x is 1 for digits 4, 5, 6, or 7.
- These conditions can be expressed by the following Boolean output functions:

CS T34-DIGITAL SYSEM DESIGN
UNIT – II (11MARKS)

Y2/S3

$$z = D_1 + D_3 + D_5 + D_7$$

$$y = D_2 + D_3 + D_6 + D_7$$

$$x = D_4 + D_5 + D_6 + D_7$$

The encoder can be implemented with three OR gates.

Truth Table of an Octal-to-Binary Encoder

Inputs								Outputs		
D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	x	y	z
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

- The encoder defined in Table has the limitation that only one input can be active at any given time.
- If two inputs are active simultaneously, the output produces an undefined combination.
- For example, if D_3 and D_6 are 1 simultaneously, the output of the encoder will be 111 because all three outputs are equal to 1.
- The output 111 does not represent either binary 3 or binary 6.
- To resolve this ambiguity, encoder circuits must establish an input priority to ensure that only one input is encoded.
- If we establish a higher priority for inputs with higher subscript numbers, and if both D_3 and D_6 are 1 at the same time, the output will be 110 because D_6 has higher priority than D_3 .
- Another ambiguity in the octal-to-binary encoder is that an output with all 0's is generated when all the inputs are 0; but this output is the same as when D_0 is equal to 1.

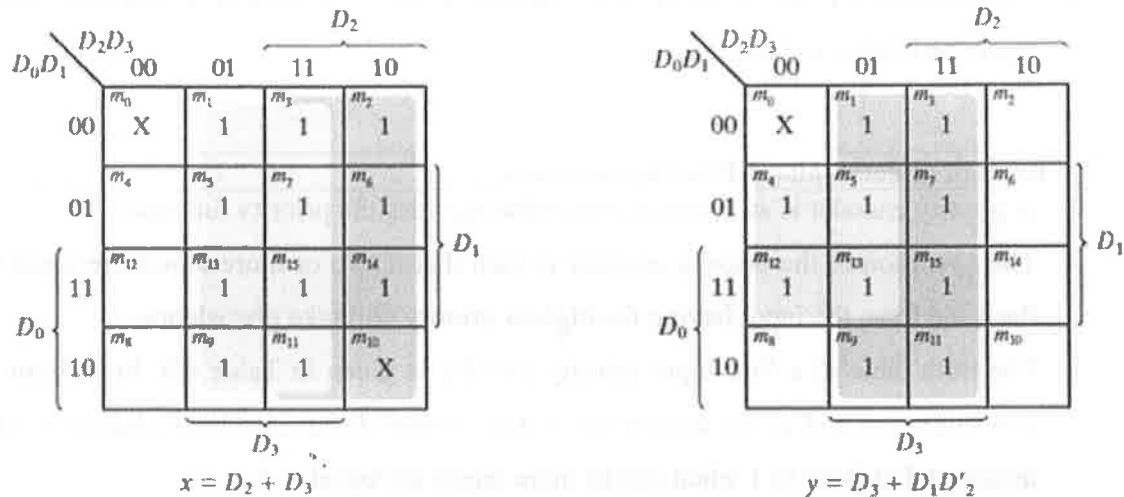
- The discrepancy can be resolved by providing one more output to indicate whether at least one input is equal to 1.

7. Explain in detail about Priority encoders

- A priority encoder is an encoder circuit that includes the priority function.
- The operation of the priority encoder is such that if two or more inputs are equal to 1 at the same time, the input having the highest priority will take precedence.
- The truth table of a four-input priority encoder is given in Table 4.8. In addition to the two outputs x and y , the circuit has a third output designated by V ; this is a *valid* bit indicator that is set to 1 when one or more inputs are equal to 1.
- If all inputs are 0, there is no valid input and V is equal to 0.
- The other two outputs are not inspected when V equals 0 and are specified as don't-care conditions.
- Note that whereas X's in output columns represent don't-care conditions, the X's in the input columns are useful for representing a truth table in condensed form. Instead of listing all 16 minterms of four variables, the truth table uses an X to represent either 1 or 0.
- For example, X 100 represents the two minterms 0100 and 1100.
- According to Table 4.8, the higher the subscript number, the higher the priority of the input. Input D_3 has the highest priority, so, regardless of the values of the other inputs, when this input is 1, the output for xy is 11 (binary 3). D_2 has the next priority level.
- The output is 10 if $D_2 = 1$, provided that $D_3 = 0$, regardless of the values of the other two lower priority inputs.
- The output for D_1 is generated only if higher priority inputs are 0, and so on down the priority levels.

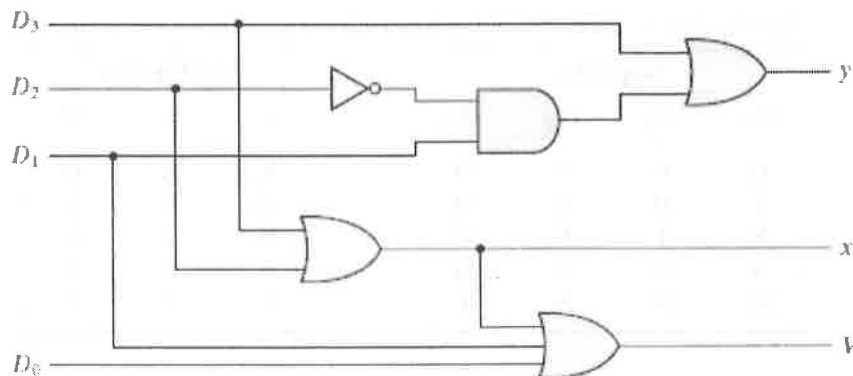
Truth Table of a Priority Encoder

Inputs				Outputs		
D_0	D_1	D_2	D_3	x	y	V
0	0	0	0	X	X	0
1	0	0	0	0	0	1
X	1	0	0	0	1	1
X	X	1	0	1	0	1
X	X	X	1	1	1	1



- The maps for simplifying outputs x and y are shown in Fig. 4.22.
- The minterms for the two functions are derived from Table 4.8.
- Although the table has only five rows, when each X in a row is replaced first by 0 and then by 1, we obtain all 16 possible input combinations.
- For example, the fourth row in the table, with inputs XX10, represents the four minterms 0010, 0110, 1010, and 1110.
- The simplified Boolean expressions for the priority encoder are obtained from the maps.
- The condition for output V is an OR function of all the input variables.
- The priority encoder is implemented in Fig. 4.23 according to the following Boolean functions:

$$\begin{aligned}
 x &= D_2 + D_3 \\
 y &= D_3 + D_1 D'_2 \\
 V &= D_0 + D_1 + D_2 + D_3
 \end{aligned}$$



CS T34-DIGITAL SYSEM DESIGN

UNIT - II (11MARKS)

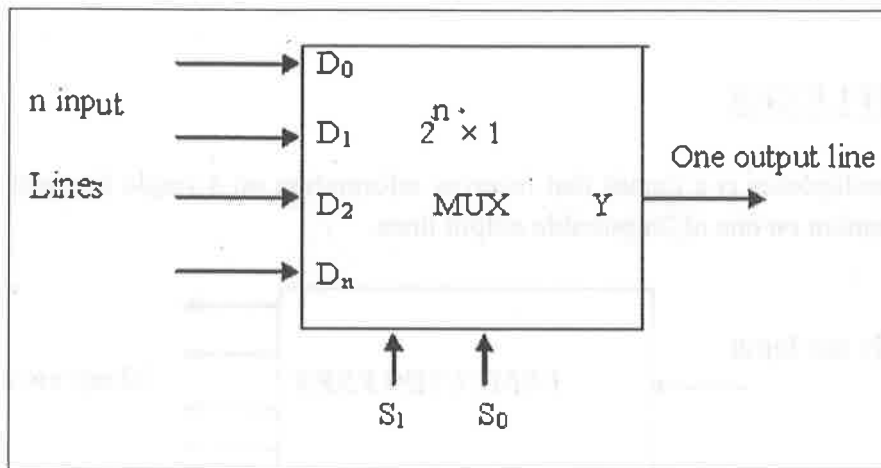
Y2/S3

8. Explain in detail the 4 to 1 line multiplexer.
 Explain Multiplexers.
 Write about Multiplexer and Demultiplexer.

(11) (NOV'11)
 (5) (NOV'11)
 (11)(APR/MAY'12)

Concept of Multiplexer:

- Multiplexer is a digital switch and it allows digital information from several sources into a single output line.



- The basic multiplexer has several data input lines and a single output line.
- The selection of a particular input line is controlled by a set of selection lines. Generally there are 2^n input lines and n selection line, whose bit combinations determine which input is selected.
- Therefore multiplexer follows many to one concept.

Example of 4 to 1 multiplexer

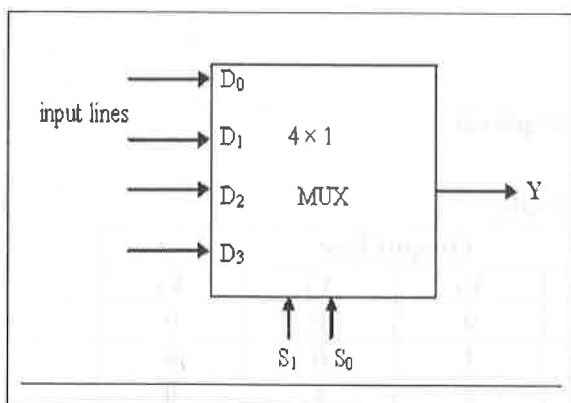


Figure: 4:1 Multiplexer

SELECTION LINES		OUTPUT
S ₁	S ₀	Y
0	0	D ₀
0	1	D ₁
1	0	D ₂
1	1	D ₃

Table: Function Table

CS T34-DIGITAL SYSEM DESIGN

UNIT - II (11MARKS)

Y2/S3

- No. of input lines $\rightarrow 4$
- No. of output lines $\rightarrow 1$
- No. of selection lines $\rightarrow 2$
- Let the input data lines are D_0, D_1, D_2 and D_3 .
- Output line is Y
- Selection lines are S_0 and S_1 .

DEMULTIPLEXER

- A demultiplexer is a circuit that receives information on a single line and transmits this information on one of $2n$ possible output lines.



- The selection of specific selection line is controlled by the values of n selection lines.
- The single input variable D_{in} as a path to all the 4 outputs.
- But the input information is directed to only one of the output at a time.

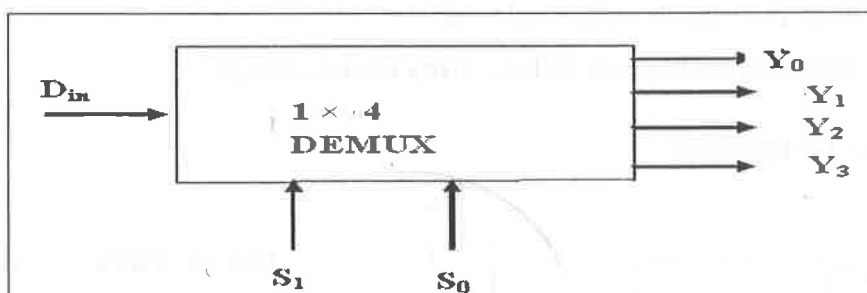


Figure: 1×4 Demultiplexer

Truth Table

Selection line		Input data	Output line			
S_1	S_0	D_{in}	Y_0	Y_1	Y_2	Y_3
0	0	1	1	0	0	0
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	1

9. Design a combination circuit that compares two 4-bit numbers. A and B, to check if they are equal. The dircuits has one output x, such that $x=1$ if $A=B$, and $x=0$ if $A \neq B$. (11)(APR'12)

Elaborate Magnitude comparator.

(11) (APR'13 - IT)

- A **magnitude comparator** is a combinational circuit that compares two numbers A and B and determines their relative magnitudes.
- The outcome of the comparison is specified by three binary variables that indicate whether
 $A > B$, $A = B$, or $A < B$.

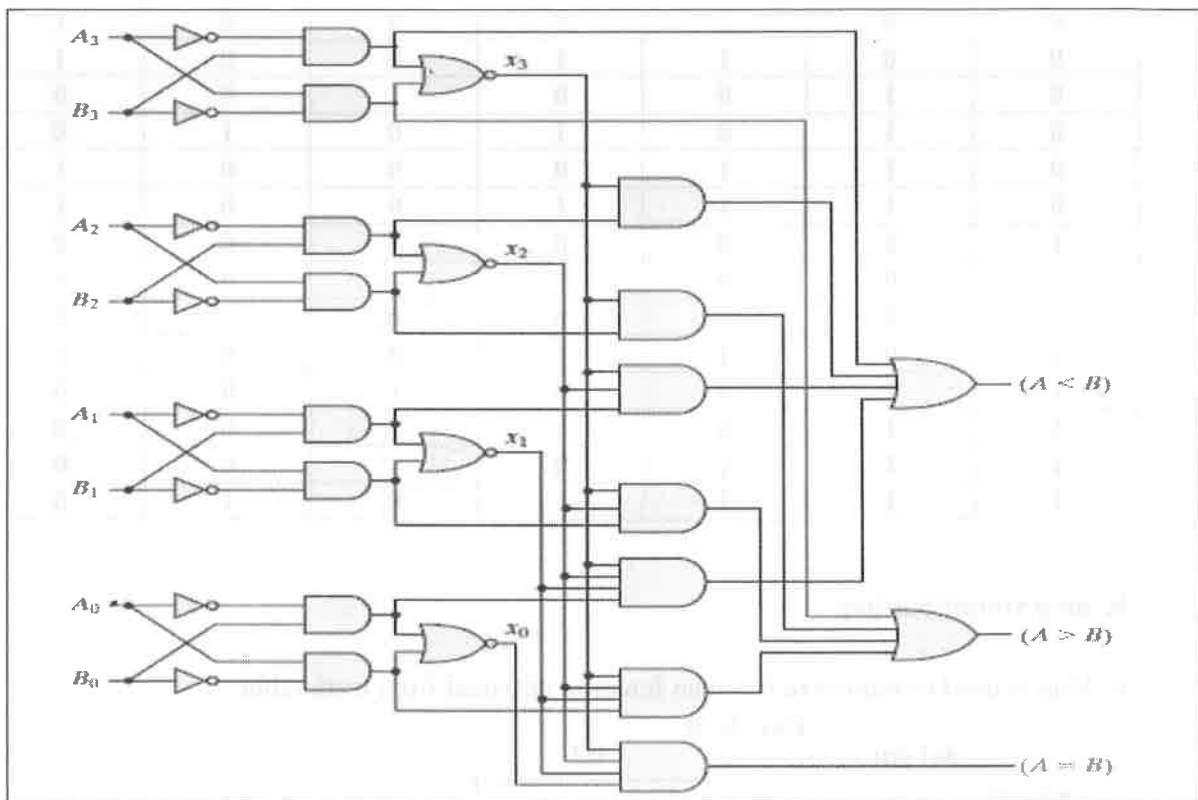


Figure: Magnitude Comparator

- the circuit for comparing two n -bit numbers has 2^n entries in the truth table
- The *binary* variable $(A = B)$ is equal to 1 only if all pairs of digits of the two numbers are equal.
- If the corresponding digit of A is 1 and that of B is 0, we conclude that $A > B$.
- If the corresponding digit of A is 0 and that of B is 1, we have $A < B$.
- The sequential comparison can be expressed logically by the two Boolean functions.

CS T34-DIGITAL SYSEM DESIGN

UNIT - II (11MARKS)

Y2/S3

$$(A > B) = A_3B_3' + x_3A_2B_2' + x_3x_2A_1B_1' + x_3x_2x_1A_0B_0'$$

$$(A < B) = A_3'B_3 + x_3A_2'B_2 + x_3x_2A_1'B_1' + x_3x_2x_1A_0'B_0$$

- The symbols $(A > B)$ and $(A < B)$ are *binary* output variables that are equal to 1 when $A > B$ and $A < B$, respectively.

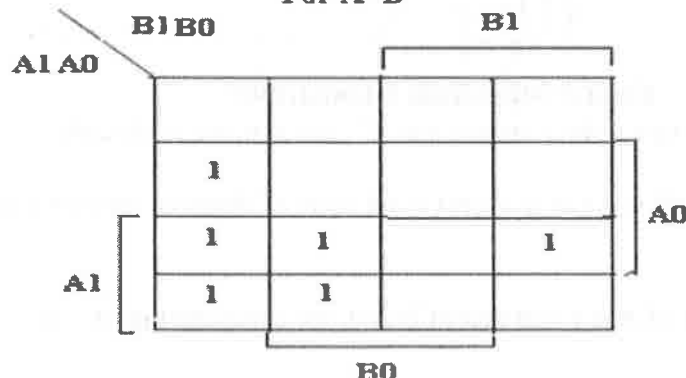
Truth table: Magnitude Comparator

INPUTS				OUTPUTS		
A1	A0	B1	B0	A>B	A=B	A<B
0	0	0	0	0	1	0
0	0	0	1	0	0	1
0	0	1	0	0	0	1
0	0	1	1	0	0	1
0	1	0	0	1	0	0
0	1	0	1	0	1	0
0	1	1	0	0	0	1
0	1	1	1	0	0	1
1	0	0	0	1	0	0
1	0	0	1	1	0	0
1	0	1	0	0	1	0
1	0	1	1	0	0	1
1	1	0	0	1	0	0
1	1	0	1	1	0	0
1	1	1	0	1	0	0
1	1	1	1	0	1	0

K-map simplification:

K-Map is used to minimize Boolean function obtained from truth table

For $A > B$



$$A > B = A_1B_1' + A_0B_0'A_1'B_1' + A_0B_0'A_1B_1$$

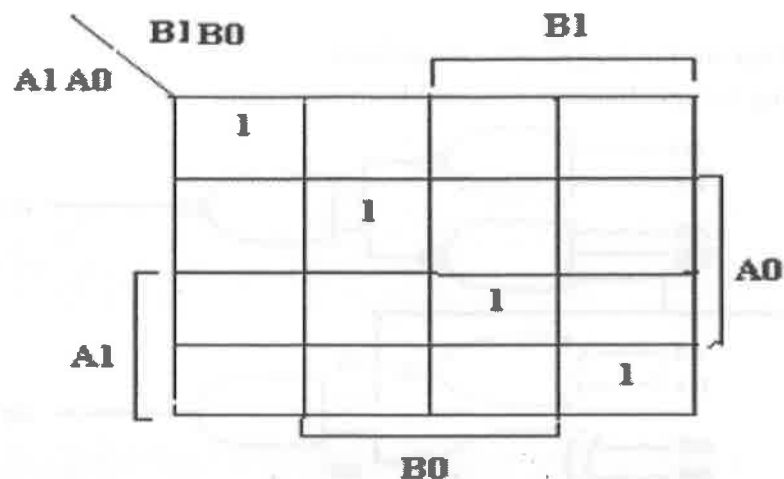
$$= A_1B_1' + A_0B_0'(A_1'B_1' + A_1B_1)$$

$$= A_1B_1' + A_0B_0' X_1$$

CS T34-DIGITAL SYSEM DESIGN
UNIT - II (11MARKS)

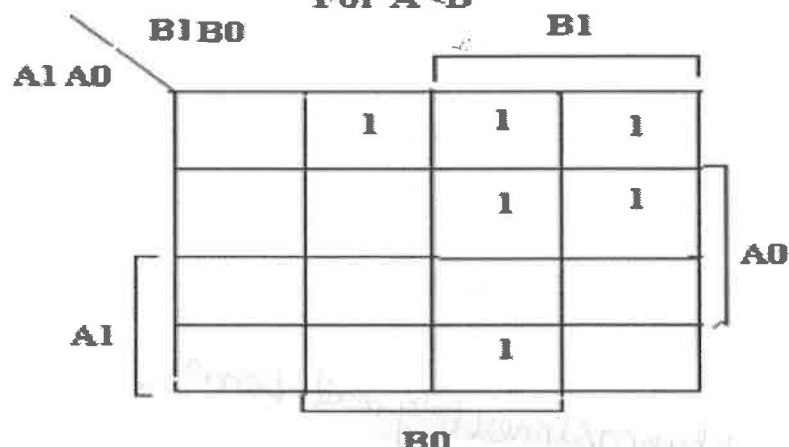
Y2/S3

For A=B



$$\begin{aligned}
 A=B: &= A1'A0'B1'B0' + A1'A0B1'B0 + A1A0'B1B0' + A1A0B1B0 \\
 &= (A1'B1' + A1B1) (A0'B0' + A0B0) \\
 &= X1X0
 \end{aligned}$$

For A<B



$$\begin{aligned}
 A<B: &= A1'B1 + A0'B0A1'B1' + A0'B0A1B1 \\
 &= A1'B1 + A0'B0(A1'B1' + A1B1) \\
 &= A1'B1 + A0'B0 X1
 \end{aligned}$$

CS T34-DIGITAL SYSEM DESIGN

UNIT - II (11MARKS)

Y2/S3

Logic Diagram for magnitude comparator

According to logic function obtained from truth table

